Appendix C: IEEE 830 Template

Software Requirements Specification

for

« GameStore Hub »

Version 4.0

Prepared by Vladyslava Maneliuk

Lodz University of Technology – IFE

March 14, 2026

Table of Contents

Revision History

Name	Date	Reason for Changes	Version
Vladyslava Maneliuk	10.03.2026	«”Initial version”»	«1.0 »
Vladyslava Maneliuk	17.03.2026	«Added UML Class, Sequence, and State diagrams to define system architecture»	«2.0 »
Vladyslava Maneliuk	24.03.2026	«Added ER Diagram and Database Schema mapping»	«3.0 »
Vladyslava Maneliuk	14.04.2026	«Added Activity Diagram for core business processes and logic flow.»	«4.0 »

1. Introduction

1.1. Purpose

This document specifies the software requirements for " GameStore Hub " version 4.0. The system is designed to provide a digital marketplace for video games, enabling users to purchase, download, and play games online.

1.2. Intended Audience and Reading Suggestions

This specification is intended for:

Developers: To implement the core marketplace and library logic.

Testers: To verify system functionality against the use cases.

Project Managers: To plan development phases and resource allocation.

End Users: To understand the basic features of the platform.

1.3. Product Scope

GameStore Hub is a self-contained digital distribution platform. Its primary goal is to bridge the gap between game developers and players by providing a secure environment for transactions, game delivery, and bug reporting.

1.4. References

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

Project Assignment Document (Lab #1).

2. Overall Description

2.1. Product Perspective

GameStore Hub is a new, standalone software system designed to emulate the core functionalities of platforms like Steam. It manages the entire lifecycle of a game from a developer's perspective (publishing) to a customer's perspective (playing).

2.2. Product Features

The major features based on the Use Case Diagram include:

Marketplace Operations: Buying games and processing refund requests.

Library Management: Saving games to a personal library and managing downloads.

Gaming: Online and offline gameplay capabilities.

Developer Tools: Adding new games to the system.

Quality Control: A unified bug reporting system connecting customers, testers, and programmers.

2.3. User Classes and Characteristics

Customer: Regular users who browse, purchase, and play games. They can also report bugs.

Developer: Professional entities that upload games and manage game-specific data.

Tester: Specialized users responsible for verifying bug reports.

Programmer: Technical staff assigned to fix reported bugs.

2.4. Operating Environment

OS: Windows 10/11, macOS 12+, Linux (Ubuntu 22.04+).

Network: Broadband internet connection for downloads and online play.

Database: Relational database for transaction and user data storage.

2.5. Design and Implementation Constraints

The system must handle concurrent downloads without performance degradation.

All financial transactions must be encrypted.

Code comments and documentation must be in English.

2.6. System Architecture

To illustrate the structural design of the GameStore Hub, the following Class Diagram defines the primary entities, their attributes, and relationships.

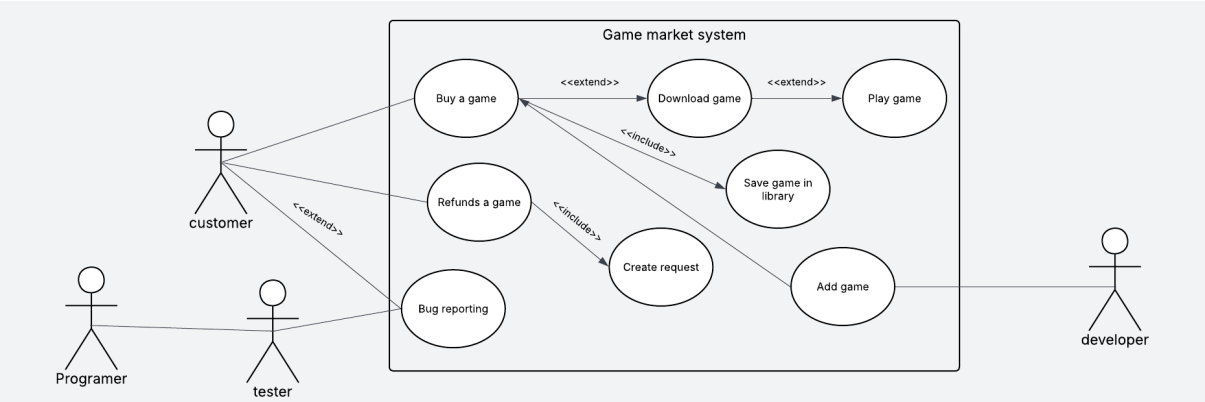

Figure 1. Use Case Diagram for GameStore Hub

This diagram defines the static structural model of the system. It showcases the inheritance hierarchy where the abstract 'User' class is specialized into four distinct roles: Customer, Developer, Tester, and Programmer. Additionally, it illustrates the relationships between users, games, orders, and the bug reporting system, including attributes and methods for each entity.

Key Class Descriptions:

- **Customer:** Manages personal balance, library, and purchasing logic.
- **Developer:** Responsible for game uploads and sales analytics.
- **Game:** Contains metadata, pricing, and versioning info.
- **Order:** Represents a unique transaction between a Customer and a Game.

3. System Features

3.1. Game Purchase and Delivery

The system allows customers to browse the store, complete a purchase, and automatically add the title to their digital library.

3.1.1. Functional Logic (Activity View)

To represent the operational workflow of the purchasing process, the following Activity Diagram outlines the decisions and actions performed by the user and the system.

- **Process Flow:** The workflow starts with browsing. If the user is authorized and the payment is valid, the system performs parallel actions: updating the transaction record and adding the game to the library.

3.2. Bug Reporting Lifecycle

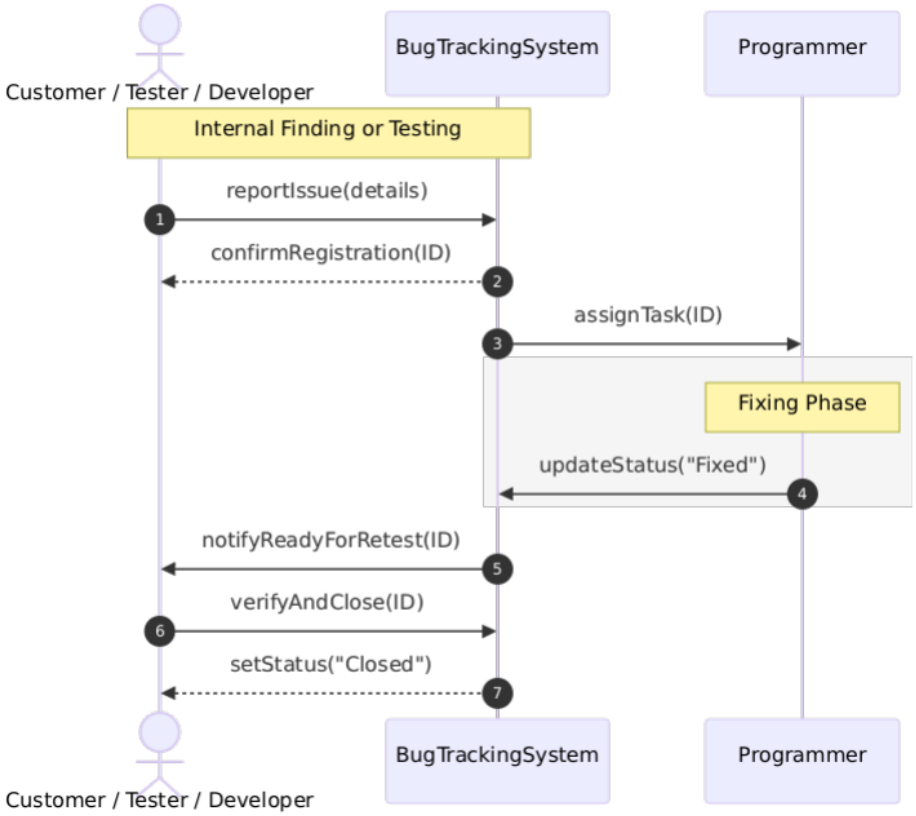

Figure 2. Sequence Diagram for Internal Bug Tracking and Verification

The following sequence diagram illustrates the communication flow as defined in the system features. It highlights how internal staff (Testers and Developers) identify issues, assign them to programmers, and verify the final fix.

3.2.1. Behavioral Logic (Sequence View)

The interaction between different user roles (Customer, Tester, Programmer) during the bug reporting cycle is illustrated in the sequence of messages below.

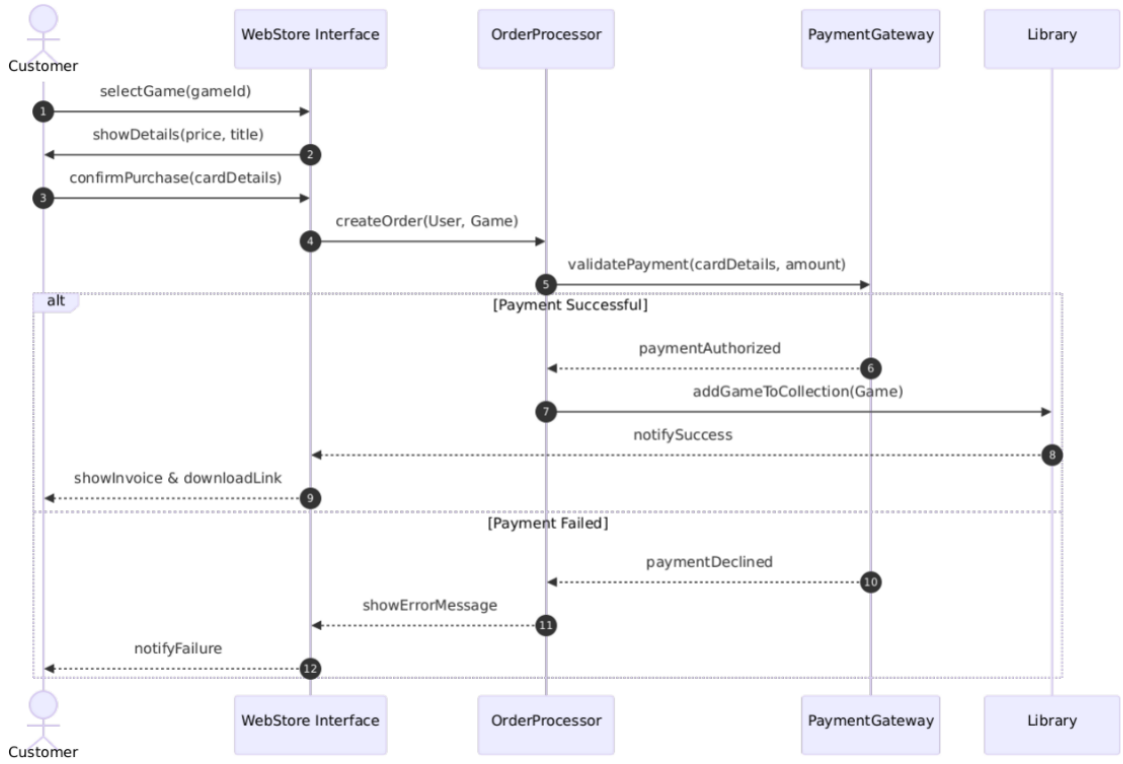

Figure 3. Sequence Diagram of the Purchase Workflow

4. External Interface Requirements

4.1. User Interfaces Overview

The system will feature a Graphical User Interface (GUI) built with modern design principles (e.g., Electron or Qt). It will include a dashboard for the library, a store page, and a dedicated bug report form.

4.3. Software Interfaces

Payment Gateway API: Integration for secure credit card and wallet processing.

File Storage System: For hosting and distributing large game installers.

4.4. Database Requirements

The system shall utilize a relational database to ensure data integrity and persistent storage. The following **Entity-Relationship Diagram (ERD)** defines the database schema, including primary keys, foreign keys, and cardinalities.

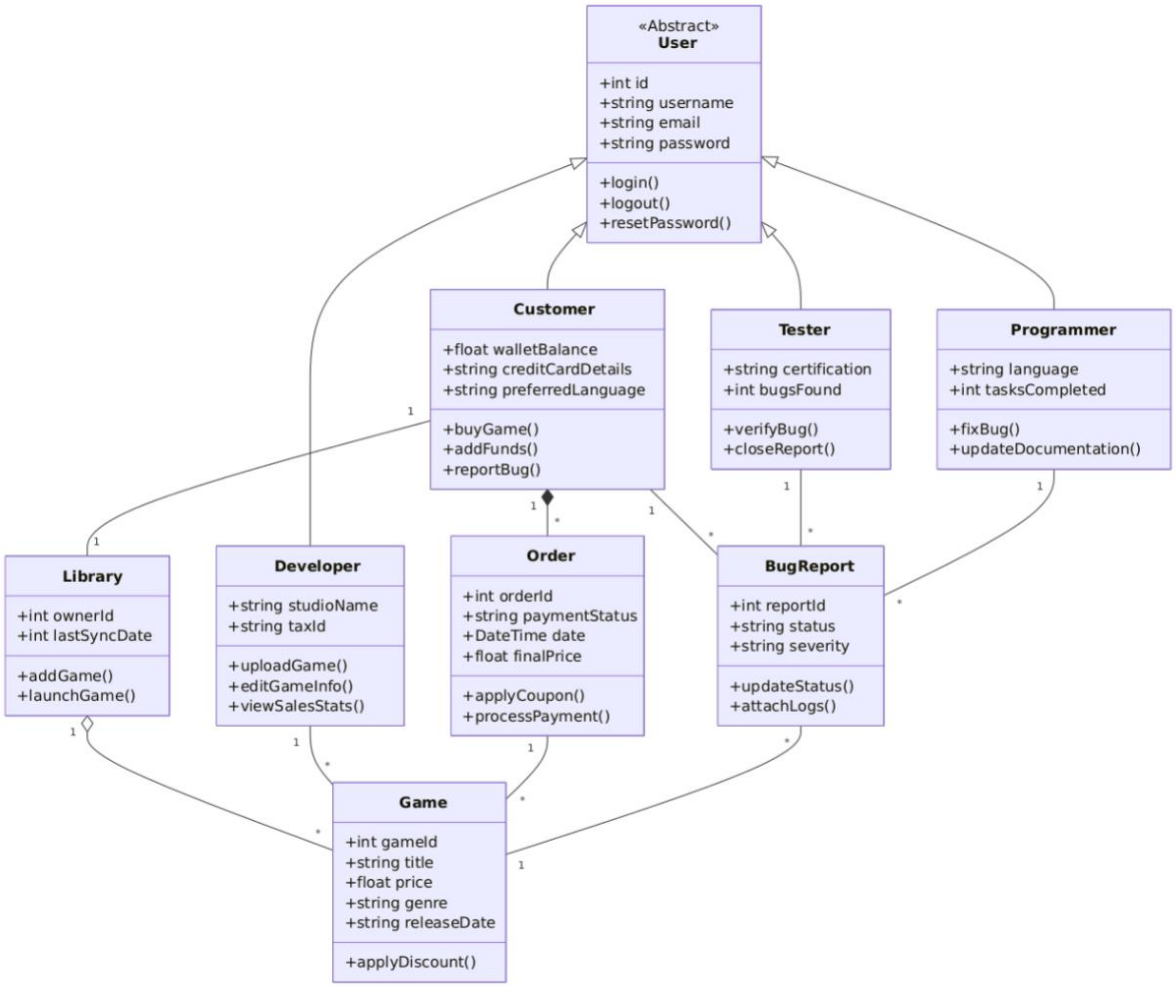

Figure 4. Entity-Relationship Diagram (ERD)

4.4.1. Data Entities Description

- **Users Table**: Stores core authentication and profile data for all system participants (id, username, email, and password_hash).
- **Games Table**: A central repository for game metadata, including titles, pricing, versions, and descriptions.
- **Developers Table**: Contains information about game creators or studios, linked to the games they publish.
- **Library Table**: Acts as a persistent collection for each user, mapping purchased games to specific user accounts for digital access.

- **Orders Table**: Records financial transactions. It serves as a link between the Customer, the Game, and the payment timestamp/status.
- **Bug Reports Table**: Stores technical issues reported within the system. It tracks the severity, current status (New, Verified, Fixed, etc.), and links the bug to a specific game and reporter.
- **User_Roles (Junction Logic)**: While represented through inheritance in classes, the database manages specific roles (Customer, Tester, Programmer) to define access levels and permissions within the platform.

5. System Features/Modules

This section details the functional requirements of the system, organized by major service modules. Each requirement is stated using "shall" to ensure it is clear, verifiable, and necessary for implementation.

5.1. Marketplace and Purchasing Module

5.1.1. Description and Priority

This module manages the game selection process, financial transactions, and communication with the payment gateway. **Priority: High**.

5.1.2. Stimulus/Response Sequences

- **Stimulus:** The Customer triggers the confirmPurchase(cardDetails) method via the user interface.
- **Response:** The system instantiates an Order object and sends a validatePayment request to the PaymentGateway.
- **Stimulus:** The PaymentGateway returns a paymentAuthorized status.
- **Response:** The system executes addGameToCollection(Game) and generates a digital invoice.

5.1.3. Functional Requirements

- **REQ-1.1:** Upon game selection, the system shall display the current price, title, and metadata using the showDetails() method.
- **REQ-1.2:** The system shall calculate the final price by applying active discounts via applyDiscount() before processing the order.
- **REQ-1.3:** The system shall prevent transaction completion if the walletBalance is insufficient or the credit card details are invalid.
- **REQ-1.4:** Upon successful payment, the system shall create a persistent link between the Customer and the Game in the Library database table.

5.2. Bug Tracking and Quality Assurance Module

5.2.1. Description and Priority

This module provides a unified mechanism for reporting, assigning, and fixing technical issues within the games. **Priority: Medium**.

5.2.2. Stimulus/Response Sequences

- **Stimulus:** A User (Customer, Tester, or Developer) calls the reportBug() method with issue details.
- **Response:** The BugTrackingSystem registers the report, assigns a unique reportId, and sets the status to "New".
- **Stimulus:** A Programmer updates the status using updateStatus("Fixed").
- **Response:** The system triggers a notifyReadyForRetest message to the original reporter.

5.2.3. Functional Requirements

- **REQ-2.1:** Every BugReport shall include mandatory attributes: severity, status, and a reference to the specific Game entity.
- **REQ-2.2:** The system shall enforce role-based access; only a Tester shall have the authority to execute the verifyBug() method.
- **REQ-2.3:** The system shall maintain a history of status changes for each report from registration to the Closed state.
- **REQ-2.4:** When a status changes to "Fixed", the system shall automatically assign a
- retesting task to the relevant quality assurance staff.

5.3. Developer Management Module

5.3.1. Description and Priority

A set of tools for game creators to publish content and monitor their business performance. **Priority: Medium**.

5.3.2. Functional Requirements

- **REQ-3.1:** The system shall provide an uploadGame() method, which requires the validation of the developer's taxId.
- **REQ-3.2:** The analytics module shall aggregate data from all Order objects to generate the viewSalesStats() report for the developer.
- **REQ-3.3:** The system shall allow developers to modify game metadata (price, genre, release date) through the editGameInfo() method.

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: The store page must load within 3 seconds under normal load.

NF-1.2: The system shall support at least 1,000 concurrent users.

6.2. Security

NF-2.1: User authentication must utilize secure hashing algorithms (e.g., Argon2 or BCrypt).